PROYECTO INTEGRADO · RA4 Y RA7

Proyecto Integrado: SafaReyes Web

Aplicación cliente en Angular para el negocio del ecosistema SafaReyes

Entrega	Modalidad	Tecnología
Frontend del proyecto	**Individual**	**Angular · TypeScript**

Partiendo del enunciado del proyecto **Backend con Spring Boot · Ecosistema SafaReyes** de Desarrollo Web en Entorno Servidor, a continuación se detalla lo que se solicitará por parte de la asignatura de DWEC: la **aplicación web en Angular** del mismo caso que tienes asignado en DWES (CoffeeStation, FitZone, CineTown o ParkControl), que será la que usen los clientes, el personal y la administración de ese negocio.

IMPORTANTE — En DWEC se evalúa el código cliente. Si tu API de DWES no estuviese terminada, la aplicación puede funcionar con datos simulados (un servidor de prueba en Express como el visto en apuntes de clase, o json-server), siempre que respete exactamente las rutas y los campos JSON del enunciado de DWES. Indica en la memoria qué partes funcionan contra la API real y cuáles contra datos simulados.

1. Estructura base en Angular

Deberá ser un proyecto **standalone** creado con Angular CLI, con todos los componentes necesarios para completar las funcionalidades de tu caso (por ejemplo: carta, detalle-producto, carrito, mis-pedidos, login, panel de administración…).

- **Modelos TypeScript** para las entidades de tu caso: interfaces que reflejen el JSON que envía y recibe la API (con los mismos nombres de campo que en DWES) y clases con comportamiento para la lógica de la aplicación (por ejemplo `Producto`, `Carrito`, `Pedido`, `Socio`, `Sesion`, `Ticket`…).
- **Servicios** para manejar la lógica y la comunicación con la API (por ejemplo `ProductoService`, `PedidoService`, `AuthService`). Los componentes no hacen peticiones HTTP directamente: se las piden a un servicio.
- **Rutas** para cada vista de la aplicación y **componentes reutilizables** que se comuniquen mediante `input()` / `output()`.
- Proyecto **organizado en carpetas** por responsabilidad. Se propone esta estructura, que puedes adaptar justificándolo en la memoria:

ESTRUCTURA DEL PROYECTO

```
src/app
├── models          → interfaces y clases del dominio (producto.ts, pedido.ts…)
├── services        → servicios de datos y de autenticación
├── components      → componentes reutilizables (tarjeta, paginador, filtro…)
├── pages           → una carpeta por vista (carta, detalle, carrito, login, admin…)
├── guards          → protección de rutas por sesión y rol
├── interceptors    → añadir el token a las peticiones
├── utils           → funciones propias (cálculos, validaciones, formateo…)
├── app.routes.ts
└── app.config.ts
src/environments    → URL de la API (desarrollo / producción)
```

2. Lógica de programación cliente (RA4)

Se valorará:

- Uso de **funciones predefinidas** del lenguaje: métodos de arrays (`map`, `filter`, `reduce`, `find`, `some`, `sort`…), de cadenas, de `Math`, `Number` y `Date`, y de `JSON` y el almacenamiento web.
- **Funciones propias** para la lógica de negocio en cliente (calcular totales, aplicar descuentos, validar un DNI o una matrícula, formatear duraciones…), documentadas y reutilizadas.
- **Gestión de arrays** para almacenar y procesar la información: inserción, eliminación, búsqueda, filtrado, ordenación y agrupación (carrito, listados, historiales…).
- **Clases** de dominio con propiedades, métodos, encapsulación y al menos un caso de **herencia y polimorfismo** justificado en tu caso.
- Uso de los **objetos definidos por el usuario** en servicios y componentes (no solo declararlos).
- **Código comentado** y estructurado.

3. Comunicación asíncrona (RA7)

Se valorará:

- Integración con el backend mediante **HttpClient** (GET, POST, PUT, PATCH y DELETE) desde los servicios.
- **Carga dinámica** de los listados y detalles, con paginación y filtros enviados como parámetros de la petición.
- **Formularios** para el alta y la edición de datos, con validación en cliente y mostrando los errores que devuelve el servidor.
- Uso de **Observable** y `subscribe()` para actualizar las vistas sin recargar la página, mostrando al usuario cuándo se está cargando, cuándo hay un error y cuándo la operación ha terminado bien.
- **Inicio de sesión** con el token JWT de la API: guardar la sesión, enviar el token en las peticiones y mostrar a cada rol solo lo que puede hacer.
- Al menos una vista que se **actualice sola** cada cierto tiempo (la pantalla destacada de tu caso).

4. Aplicación final, documentación y exposición

Se valorará el atractivo y la funcionalidad de la aplicación en general, el uso de una librería de componentes (Angular Material o Bootstrap) de forma coherente, posibilidad de carga de imágenes (Cloudinary, Firebase Storage, Supabase Storage, AWS S3…) en la creación de items desde la administración (si aplica a tu caso), el vídeo explicando el trabajo realizado y la documentación entregada.

Como mejora, se valorará incluir un **widget del complejo**: un bloque que muestre en directo datos públicos de la API de un compañero con otro caso (por ejemplo, la cafetería mostrando las sesiones de cine de hoy, o el gimnasio mostrando las plazas libres del parking).

5. Requisitos funcionales

Los requisitos funcionales y las reglas de negocio son los de tu caso en el documento de DWES. Desde el cliente, la aplicación deberá ofrecer como mínimo estas vistas y consumir los 16 endpoints de negocio de tu caso, además del registro y el inicio de sesión:

Caso	Cliente / público	Personal	Administración
CoffeeStation	Carta con filtros, detalle con alérgenos, carrito con cupón, mis pedidos y seguimiento del turno	**Tablero de barra** con los pedidos por estado; disponibilidad de productos y categorías	Gestión de carta y cupones; informes de ventas
FitZone	Planes, actividades del día y reservas; mi ficha y mis cuotas	**Terminal del torno** con aforo; alta y búsqueda de socios; cobro de cuotas y morosos	Activar socios, generar cuotas; informe de ingresos
CineTown	Cartelera con filtros, sesiones por fecha, **compra de entradas**, mis compras y cancelación	Venta en taquilla y consulta de compras	Gestión de películas y programación de sesiones; informes
ParkControl	Disponibilidad por planta y tipo; tarifa vigente; mis tickets (abonado)	Barrera de entrada, mapa de plazas, **cajero** con importe en curso y tickets	Plazas, tarifas y abonados; informes

La vista marcada en negrita es la **pantalla destacada** de cada caso: es la que debe actualizarse sola (requisito del apartado 3).

Requisitos de desarrollo

Categoría	Descripción
Programación en cliente	- Implementación de funciones y métodos en componentes y servicios - Uso de funciones predefinidas del lenguaje (map, filter, reduce, etc.) - Creación de funciones propias para la lógica de negocio en cliente
Estructuras de datos	- Gestión de arrays para almacenar y procesar información (productos, carrito, socios, sesiones, tickets…) - Operaciones sobre colecciones: inserción, eliminación, filtrado, búsqueda, ordenación y agrupación
POO en TypeScript	- Interfaces para los datos de la API y clases para las entidades del dominio - Propiedades, métodos, encapsulación, herencia y polimorfismo - Uso de objetos definidos por el usuario en la lógica de la aplicación
Comunicación asíncrona	- Consumo de los endpoints del backend con `HttpClient` (GET, POST, PUT, PATCH, DELETE) - Uso de Observables (`subscribe`, `pipe`) y gestión de errores en las peticiones - Envío de datos en JSON y como parámetros de la URL (paginación y filtros)
Actualización dinámica	- Refresco de la interfaz al modificar datos (carrito, pedidos, reservas, compras, tickets…) - Uso correcto de binding, `@if` / `@for`, `input()` / `output()` y formularios - Vista con actualización periódica
Seguridad en cliente	- Registro e inicio de sesión contra la API - Envío del token en las peticiones y cierre de sesión al caducar - Rutas y opciones de menú según el rol
Depuración y documentación	- Comentarios claros en el código y organización de archivos - Uso de las herramientas de depuración del navegador / Angular DevTools - Mensajes al usuario ante errores de las peticiones

Requisitos no funcionales

Categoría	Descripción
Eficiencia en cliente	Carga fluida de los listados y operaciones sin recargas completas de la página.
Limpieza de código	Nombres adecuados, modularidad, uso de servicios y separación de responsabilidades.
Accesibilidad y UX	Interfaz clara y adaptable, con indicación visual de carga, error y éxito en las operaciones.
Compatibilidad	Funciona correctamente en Chrome y Firefox como mínimo.

6. Evidencias de aprendizaje asociadas a los CE

CE	Evidencia observable
4.a – 4.b (Funciones)	Funciones predefinidas y funciones propias en servicios, componentes y utilidades para gestionar los datos (totales, descuentos, validaciones, formateo…).
4.c – 4.d (Arrays)	Código donde se gestionan colecciones en cliente: carrito, listados, filtros, agrupaciones, historiales.
4.e – 4.h (POO en cliente)	Archivos `.ts` con interfaces y clases de dominio, herencia y polimorfismo, e instanciación y uso de esos objetos en la aplicación.
4.i (Depuración y documentación)	Comentarios JSDoc/TSDoc, mensajes de consola controlados, manejo de errores y estructura de proyecto clara.
7.a – 7.b (Análisis de la asincronía)	Memoria técnica justificando el uso de HTTP y Observables, sus ventajas frente a la comunicación síncrona y los mecanismos disponibles.
7.c – 7.f (HttpClient y Observables)	Servicios Angular consumiendo la API REST, envío de JSON y parámetros, gestión de la sesión y actualización dinámica de la interfaz.
7.g (Compatibilidad)	Evidencia de pruebas en al menos dos navegadores (capturas en la memoria o en la exposición).
7.h – 7.i (Uso y depuración de librerías)	Uso de HttpClient, RxJS y la librería de componentes, manejo de errores y análisis de las librerías utilizadas.

7. Planificación orientativa

AUTOEVALUACIÓN — Esta tabla no se entrega ni se corrige: sirve para que compruebes si vas bien de tiempo. Solo se evalúa la entrega final. La columna central indica lo que se irá viendo en clase y que puedes aplicar en esa parte del proyecto.

Semana	Al terminar la semana deberías tener…
26 – 30 oct	El proyecto creado, la estructura de carpetas y la lista de vistas y rutas de tu caso
2 – 6 nov	Los modelos (interfaces y clases) y las funciones propias de tu caso
9 – 13 nov	Las vistas principales navegables, con datos de prueba
16 – 20 nov	Los listados y detalles cargando datos de la API (real o simulada)
23 – 27 nov	Los formularios de alta y edición y el flujo principal de tu caso
30 nov – 4 dic	Login, registro y la aplicación adaptada a cada rol
9 – 11 dic	La aplicación conectada a tu API y la pantalla destacada actualizándose sola. Entrega Final

8. Entrega asociada a DWEC

La entrega deberá incluir:

- **Código fuente completo** en un repositorio de GitHub: componentes, servicios, modelos y plantillas, con un README que explique cómo levantar el proyecto (y la API o los datos simulados que necesite).
- **Memoria del proyecto** que incluya:
  - Evidencias de pruebas de comunicación cliente-servidor (pestaña Red del navegador).
  - Capturas o explicación de las herramientas de depuración utilizadas.
  - Pruebas en al menos dos navegadores distintos.
  - Evaluación de ventajas e inconvenientes de usar comunicación asíncrona y de los mecanismos disponibles.
  - Clasificación y análisis de librerías que permitan actualizaciones dinámicas.
  - Justificación de las decisiones técnicas (estructura, clases, dónde se guarda la sesión…).
- **Vídeo** grabado explicando la aplicación funcionando con los distintos roles, explicando las diferentes pantallas. En caso de dudas, en clase se podrá pedir explicar o modificar en directo alguna parte del código.

La nota del proyecto dependerá en un **25 %** de la documentación, presentación, vídeo y aplicación en general, mientras que el **75 %** restante será la nota obtenida en los CE puros.

La nota final de cada CE será la nota obtenida en dicho CE (75 %) más la nota de la documentación, presentación, exposición en vídeo y aplicación en general (25 %), para considerar la forma de presentar y defender los CE.

AUTORÍA — Debes ser capaz de explicar y modificar todo el código que entregas. La parte que no sepas explicar ante una posible defensa no se tendrá en cuenta para los CE correspondientes. Se aplica la normativa del módulo sobre copia, plagio y uso de IA.